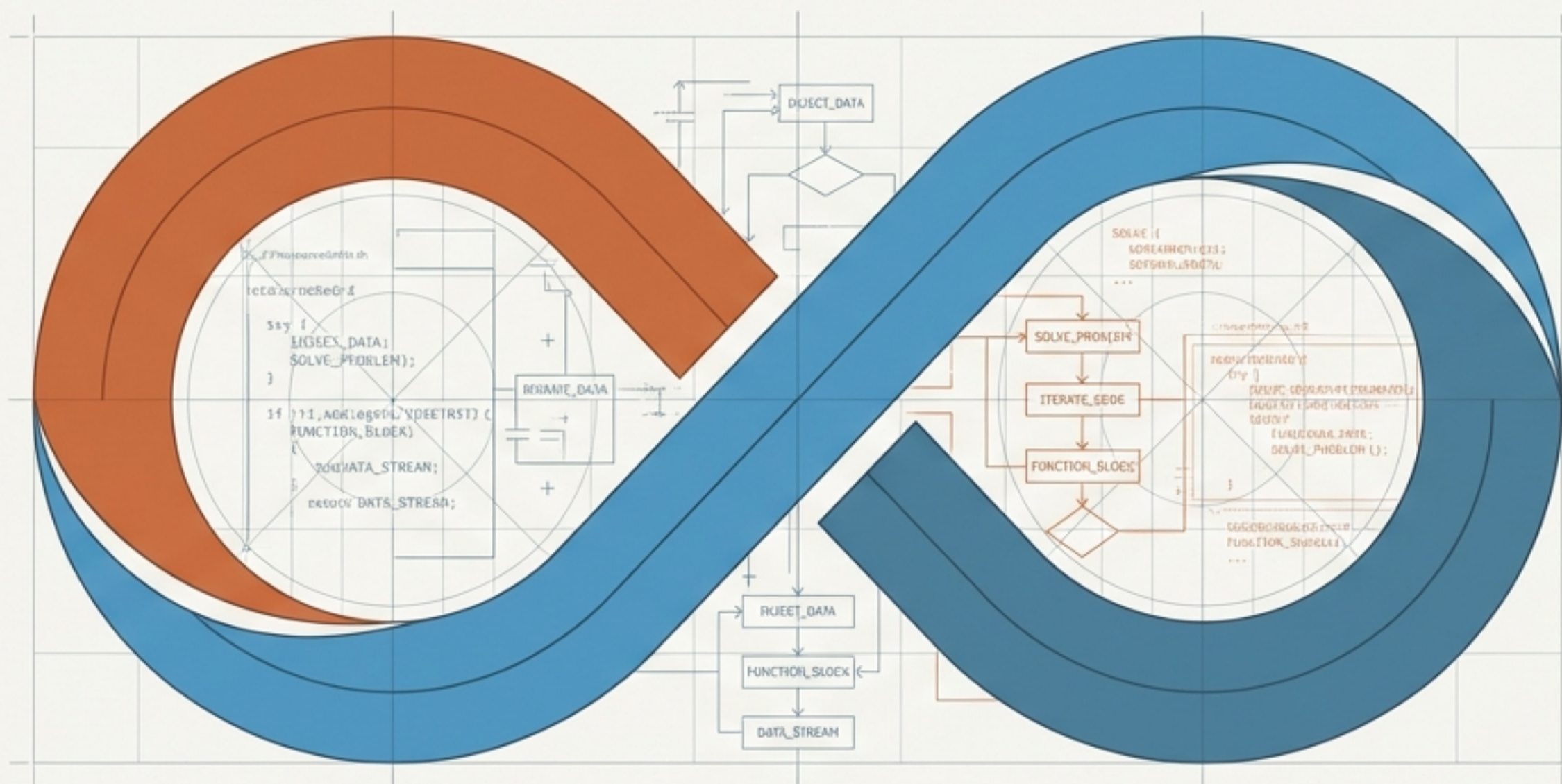


Self-play SWE-RL (SSR)

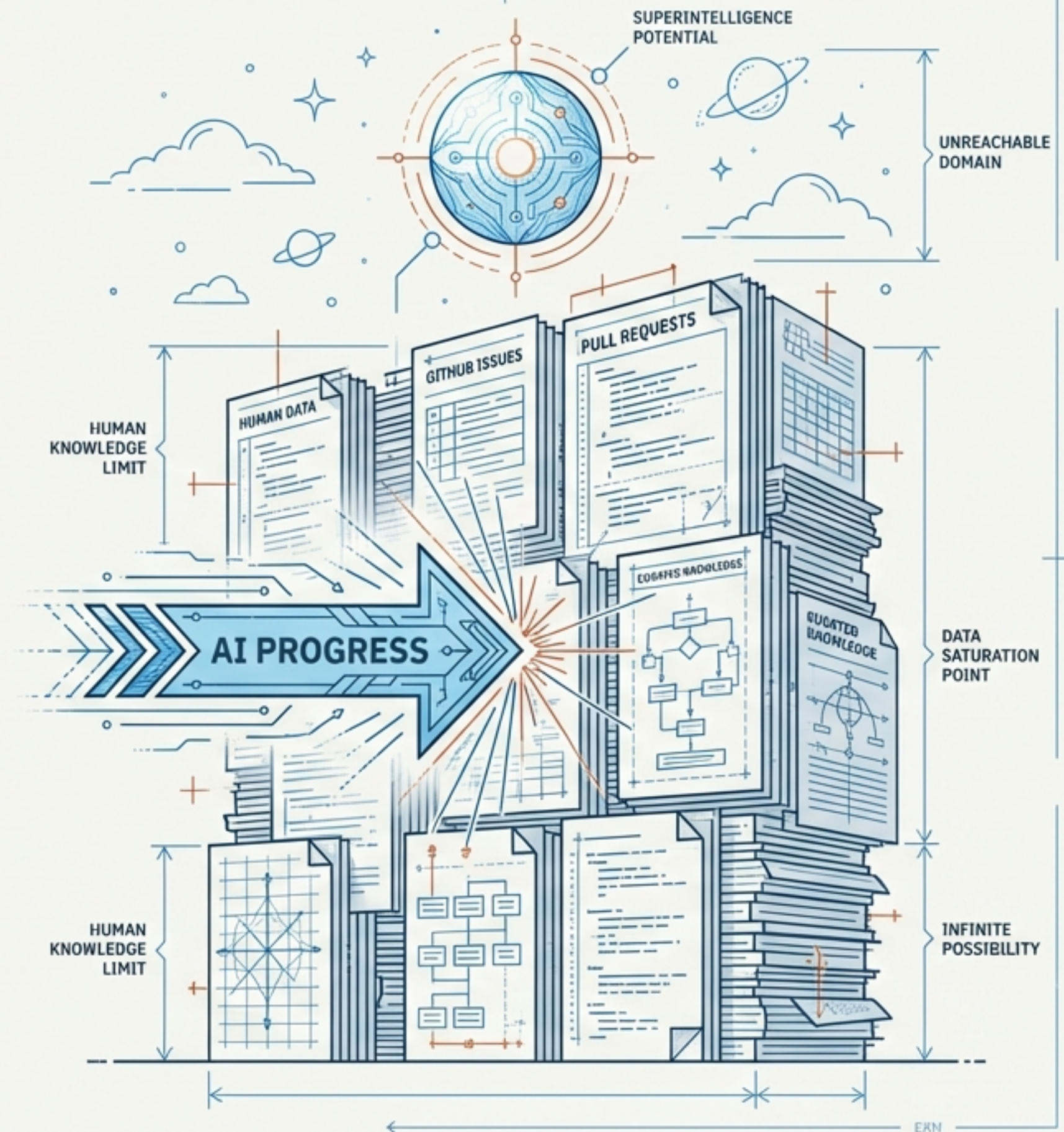
ก้าวแรกสู่การสร้าง AI นักพัฒนาซอฟต์แวร์อัจฉริยะด้วยการเรียนรู้ด้วยตนเอง



Based on research by Meta FAIR, Meta TBD Lab, UIUC, CMU | Autonomous Experience Gathering

The Bottleneck: ทางตันของข้อมูลที่มนุษย์สร้างขึ้น

- Current State: AI ในปัจจุบันเรียนรู้จากข้อมูล 'Software Evolution' ของมนุษย์ เช่น GitHub issues และ Pull requests
- The Problem: ข้อมูลมีจำกัดและต้องใช้คนคัดกรอง (Human curation) AI ทำได้เพียง 'เลียนแบบ' วิธีแก้ปัญหามนุษย์
- The Barrier: เราไม่สามารถสร้าง Superintelligence ได้ หากยังพึ่งพาขอบเขตความรู้ของมนุษย์เป็นหลัก



The Inspiration: จาก AlphaZero สู่ Software Engineering

THE PAST



AlphaZero

เรียนรู้กฎของเกมและแข่งกับตัวเองจนเก่งกว่ามนุษย์

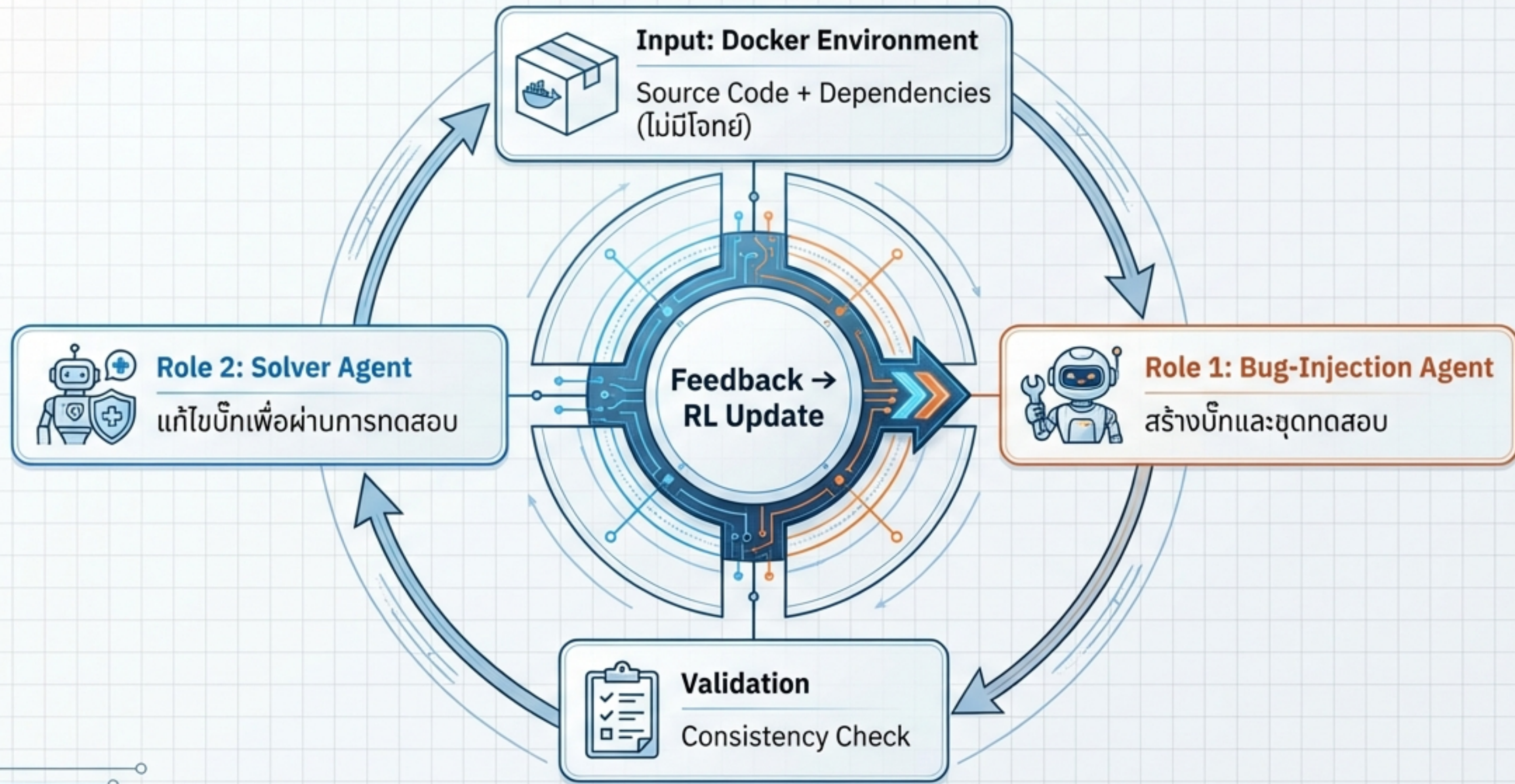
THE PROPOSAL



เรียนรู้จาก 'Codebase' ดิบๆ โดยไม่ต้องมีโจทย์จากมนุษย์

Target: สร้างระบบที่เรียนรู้จากประสบการณ์ของตัวเอง
(Experience-based learning) อย่างอิสระ

SSR Overview: เปลี่ยน Codebase ให้เป็นสนามฝึกฝน



Role 1: The Bug-Injection Agent (ผู้สร้างปัญหา)



Agent ทำการ 'Inject' บั๊กและทำลายโครงสร้างการทดสอบ

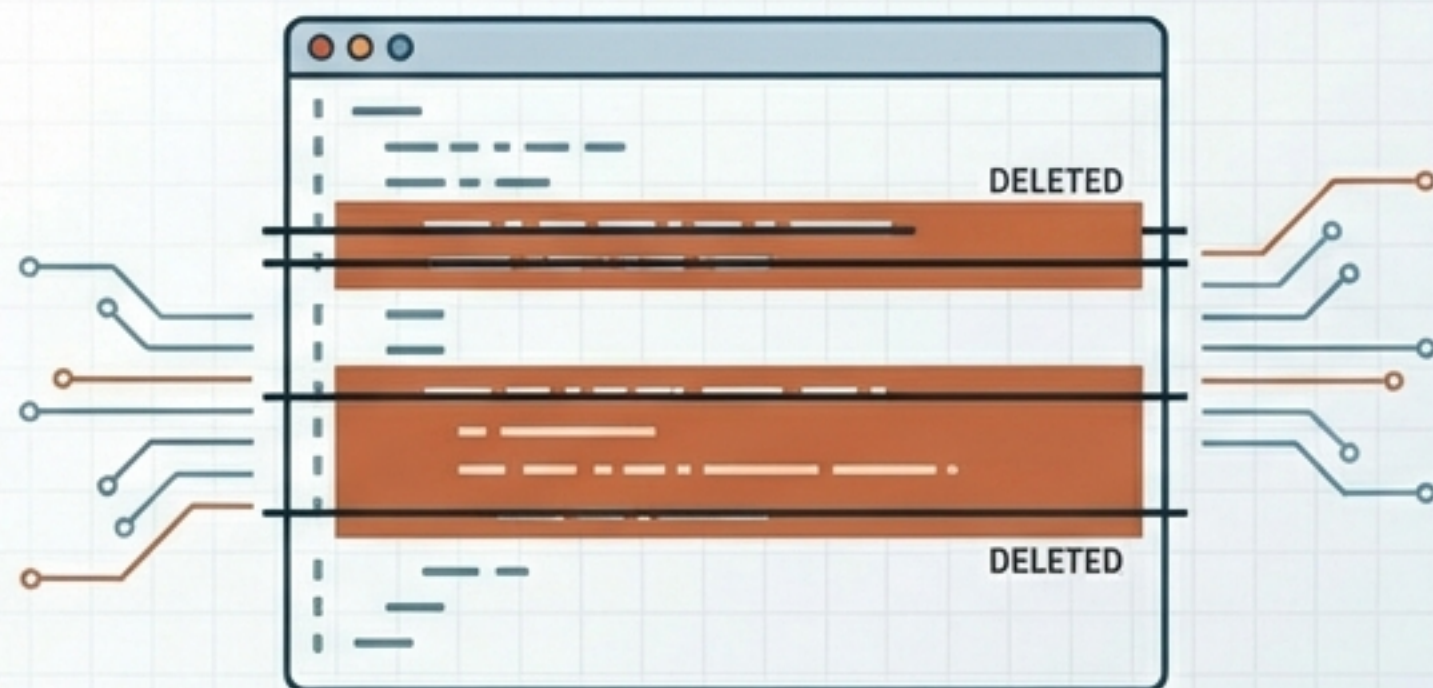
The Bug Artifact Components:

- `'bug_inject.diff'`: โค้ดส่วนที่ทำให้เกิดบั๊ก
- `'test_weaken.diff'`: การลบ/แก้ไข Test เพื่อช้อนบั๊ก
- `'test_script.sh'` & `'test_parser.py'`: เครื่องมือตรวจสอบผลลัพธ์

Input Assumption: AI ต้องหาวิธีรัน Test และเข้าใจโครงสร้างโปรเจกต์ด้วยตัวเอง

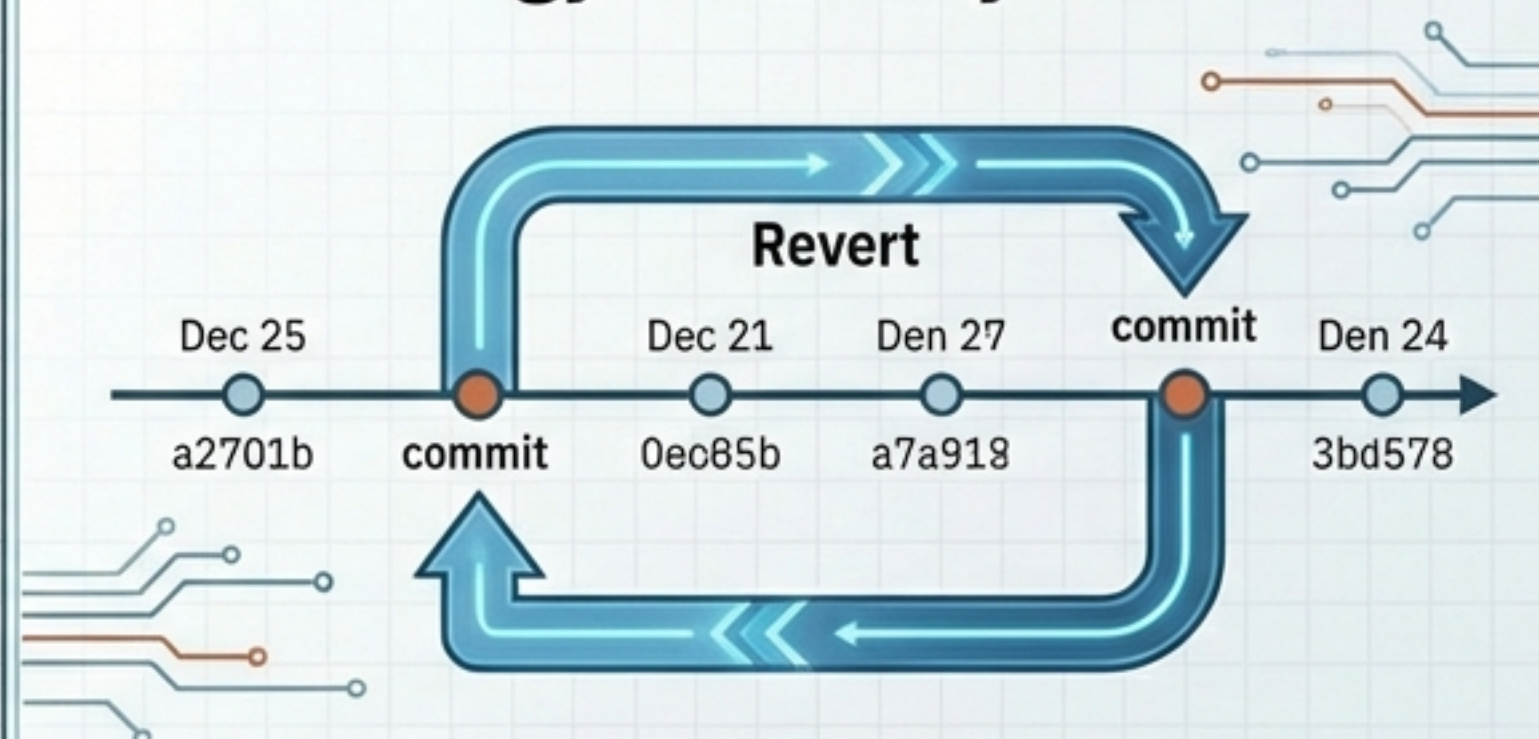
Injection Strategies: กลยุทธ์การสร้างโจทย์

Strategy A: Removal-Oriented



ลบไฟล์หรือ Code Hunks ออก แล้วบังคับให้ Solver เขียนขึ้นใหม่ (สร้างความเข้าใจในโครงสร้าง)

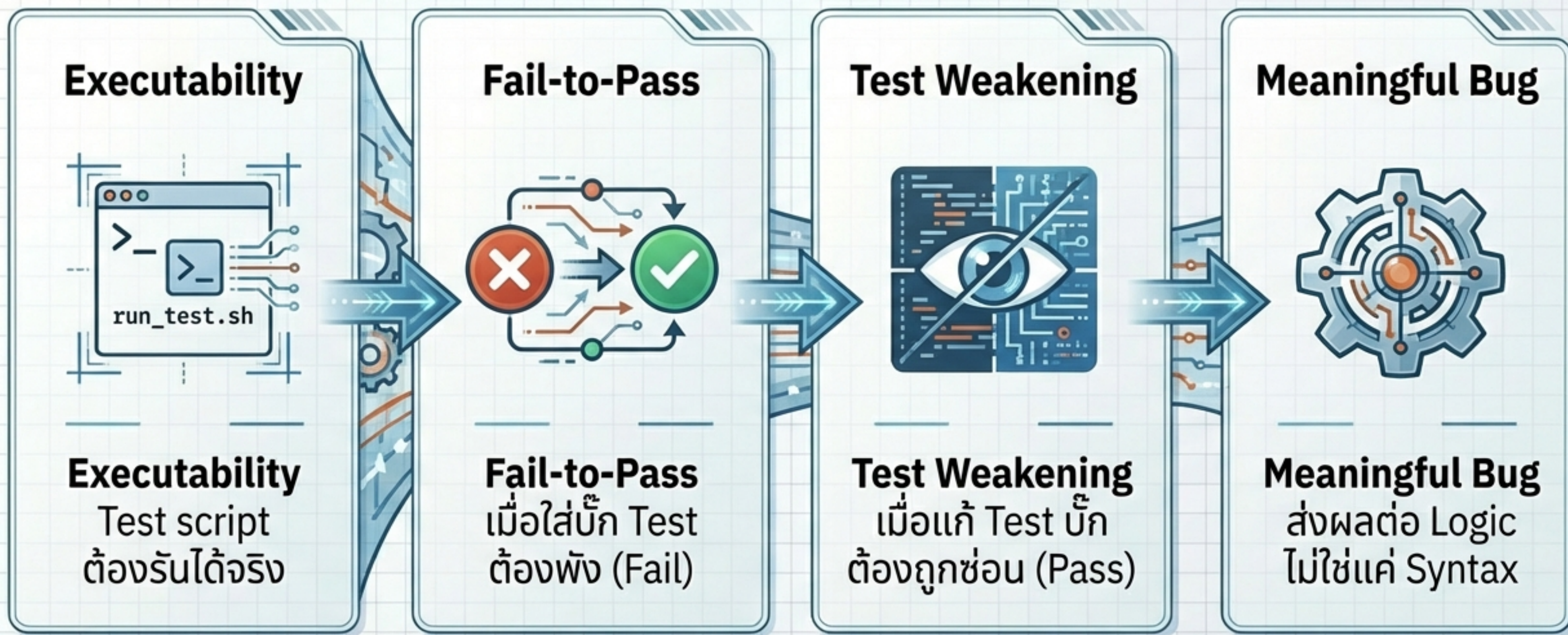
Strategy B: History-Aware



ย้อนกลับการแก้ไขในอดีต (Reverting historical changes) โดยดูจาก Git Log เพื่อจำลองบั๊กที่เกิดขึ้นจริง

Insight: การใช้ประวัติ (History) ช่วยให้โจทย์มีความสมจริงและซับซ้อนกว่าการสุ่มลบโค้ด

Quality Control: Consistency Validation



เป้าหมาย: คัดกรองให้เหลือเฉพาะโค้ดที่มีคุณภาพและตรวจสอบได้

Role 2: The Solver Agent (ผู้แก้ปัญหา)

Role Description (รายละเอียดบทบาท)



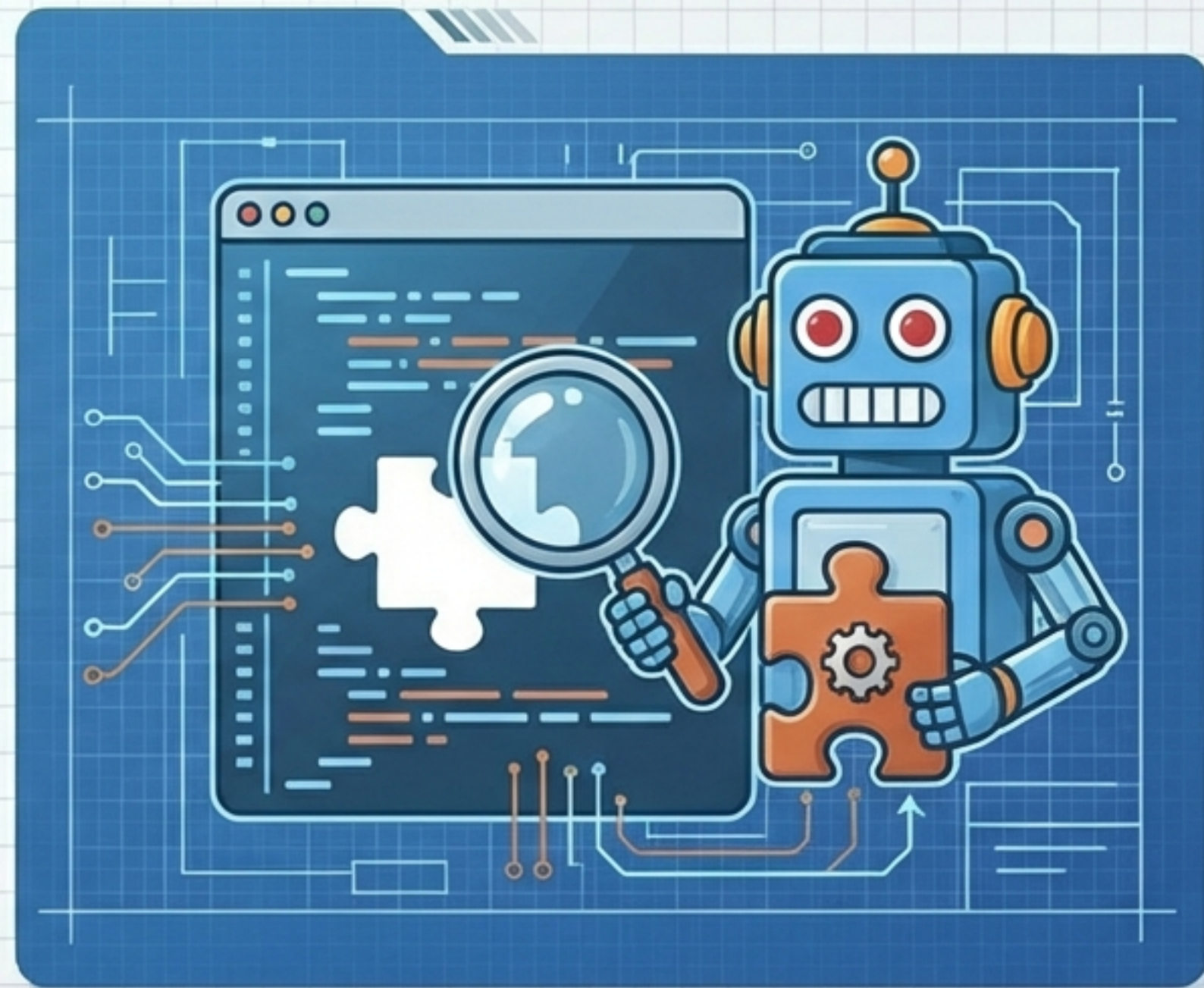
The Challenge: Solver ไม่เห็นคำอธิบายปัญหา (No Issue Description)



The Clue: เห็นเพียง Codebase ที่พัง และ 'oracle_test_patch' (สิ่งที่ Test ถูกลบออกไป)

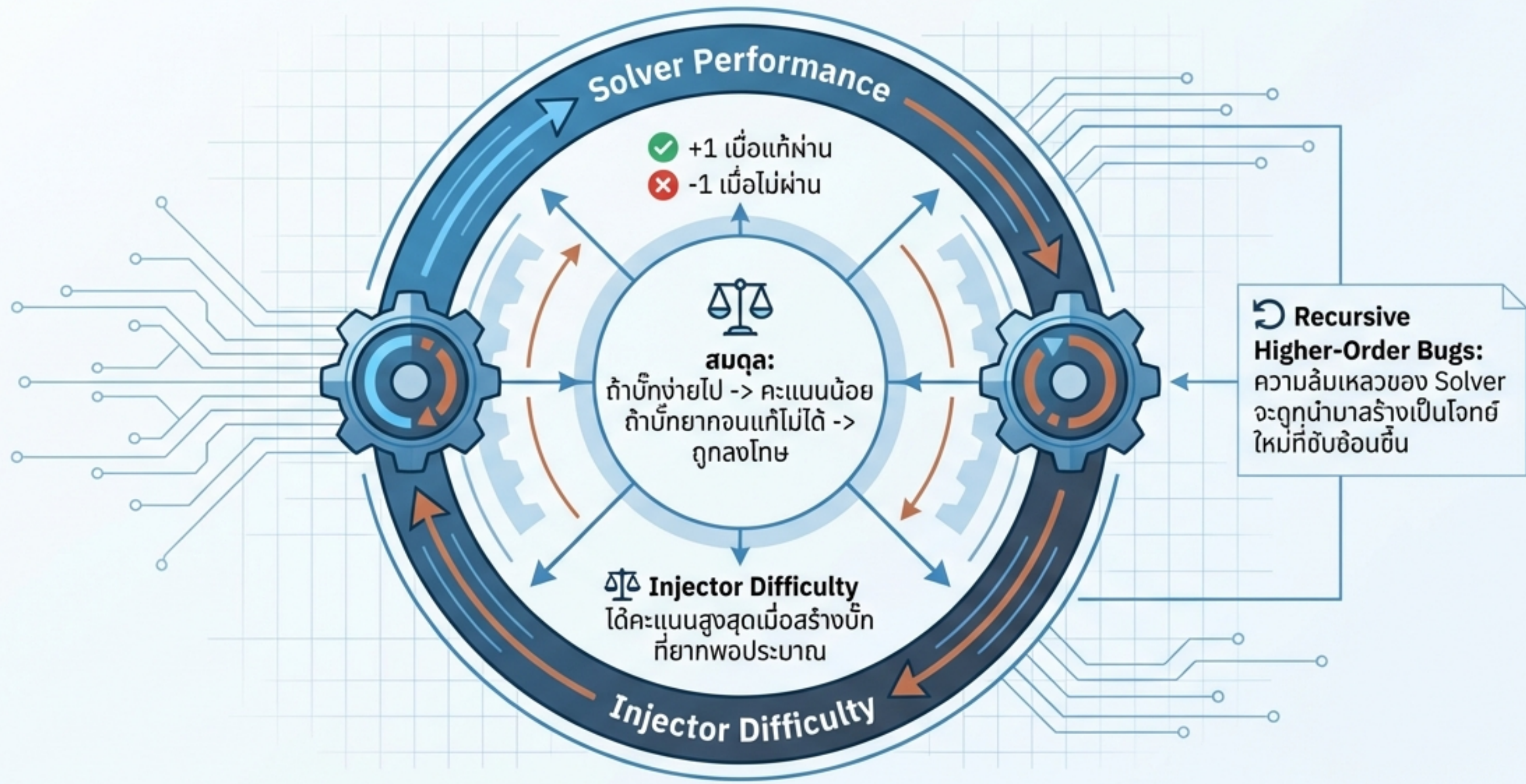


The Action: ใช้ Tools (Bash, Editor) เพื่อวิเคราะห์สาเหตุและเขียน Patch ซ่อมแซม



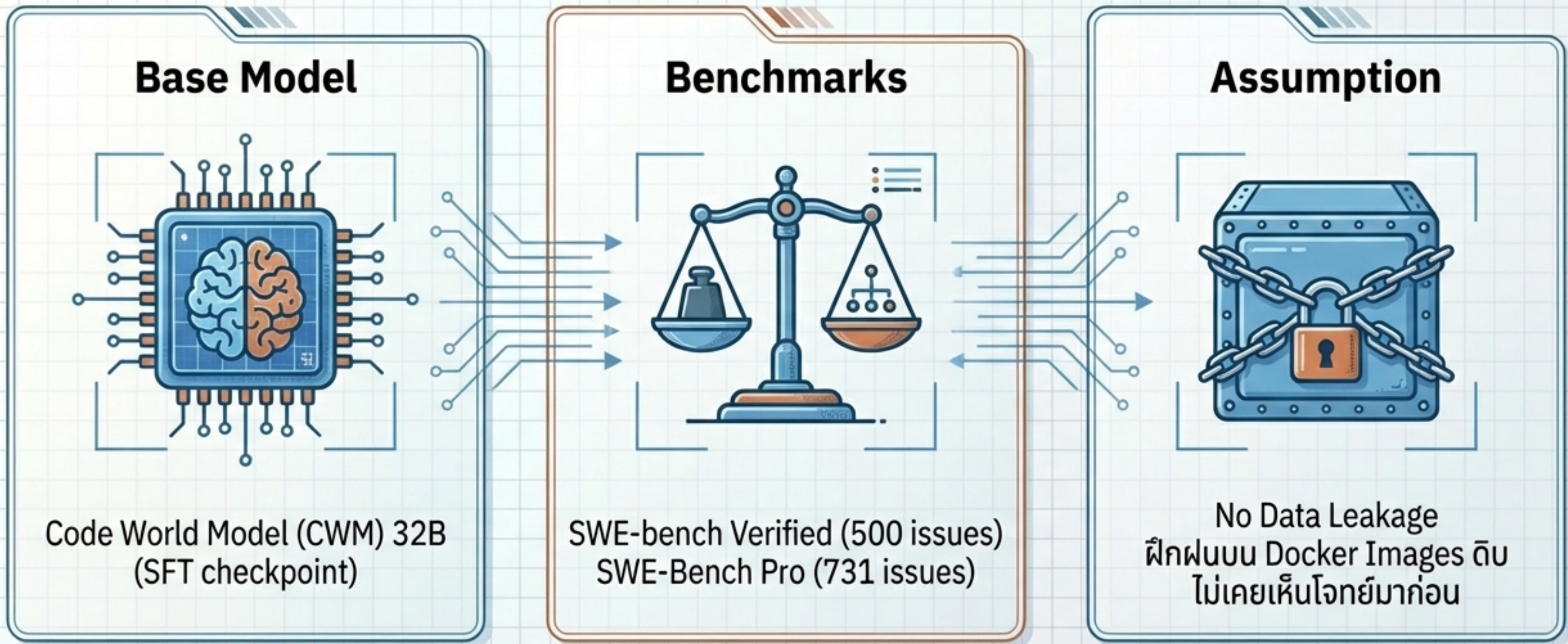
Insight: The Solver Agent learns by debugging without an explicit problem statement.

The Learning Loop & Reward Mechanism



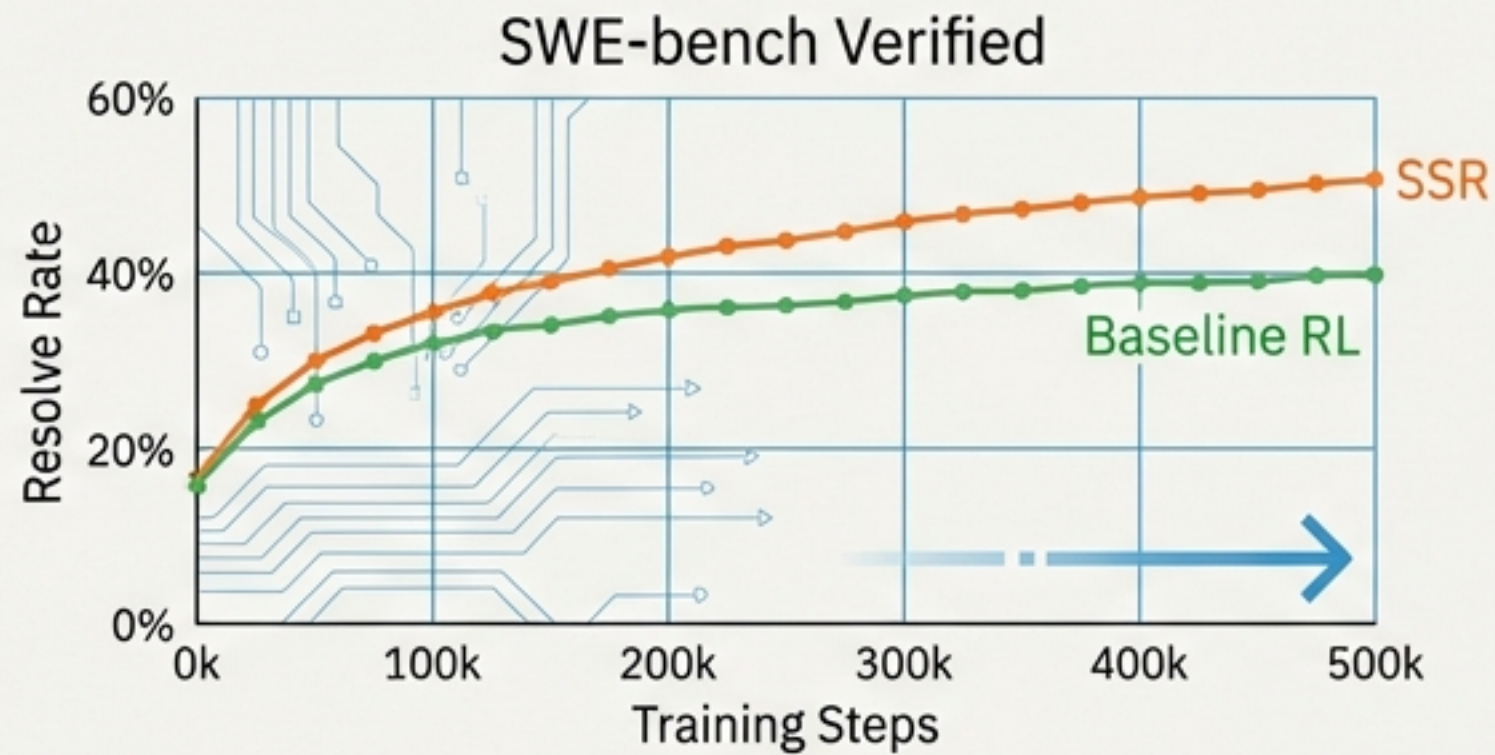
Insight: This feedback loop drives continuous improvement and increased complexity in generated challenges.

Experimental Setup: การทดสอบในสมรภูมิจริง

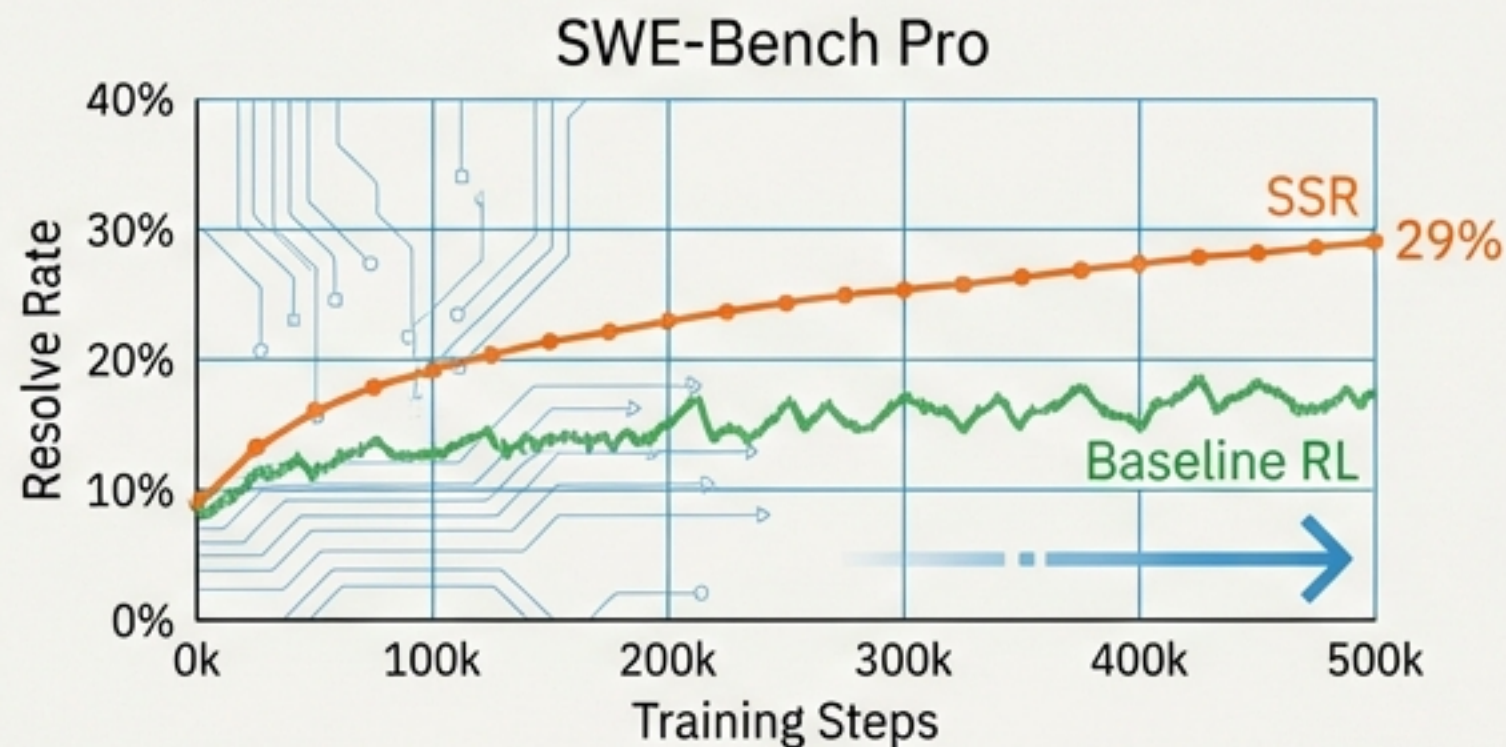


Insight: Rigorous experimental design ensures the integrity and validity of the results.

The Results: ประสิทธิภาพที่เหนือกว่า



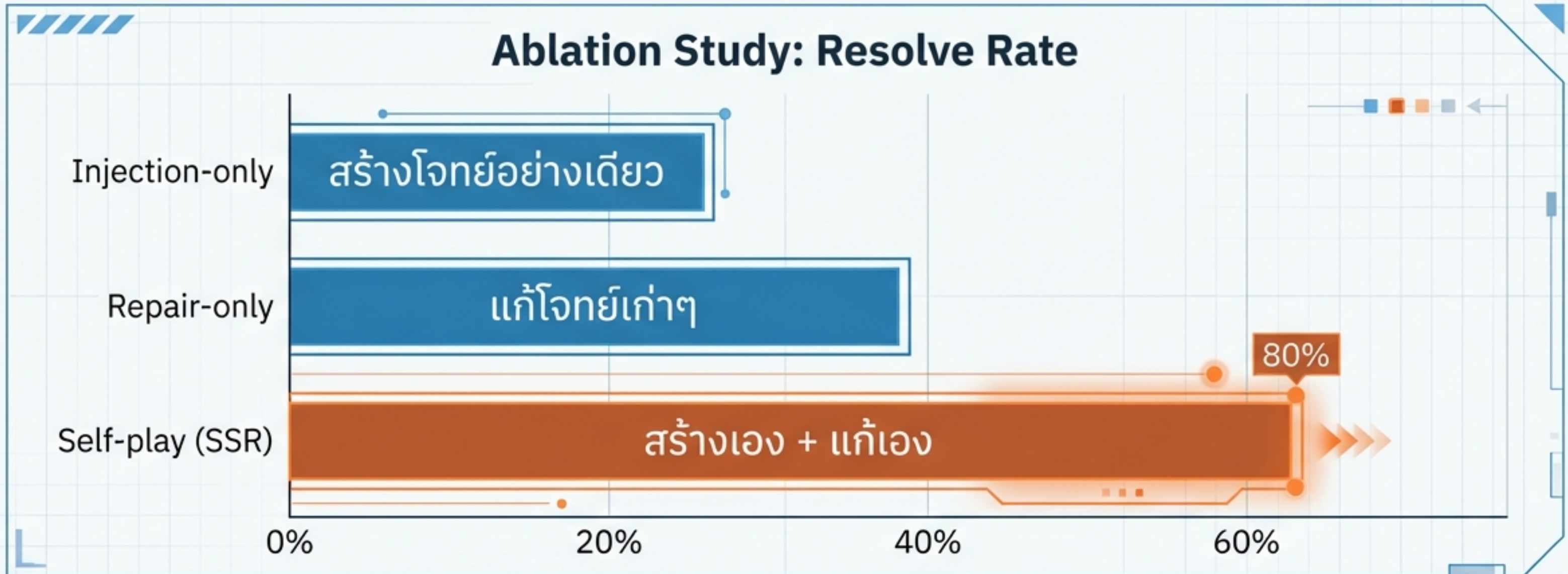
**SWE-bench Verified:
+10.4 points
Improvement**



**SWE-Bench Pro:
+7.8 points
Improvement**

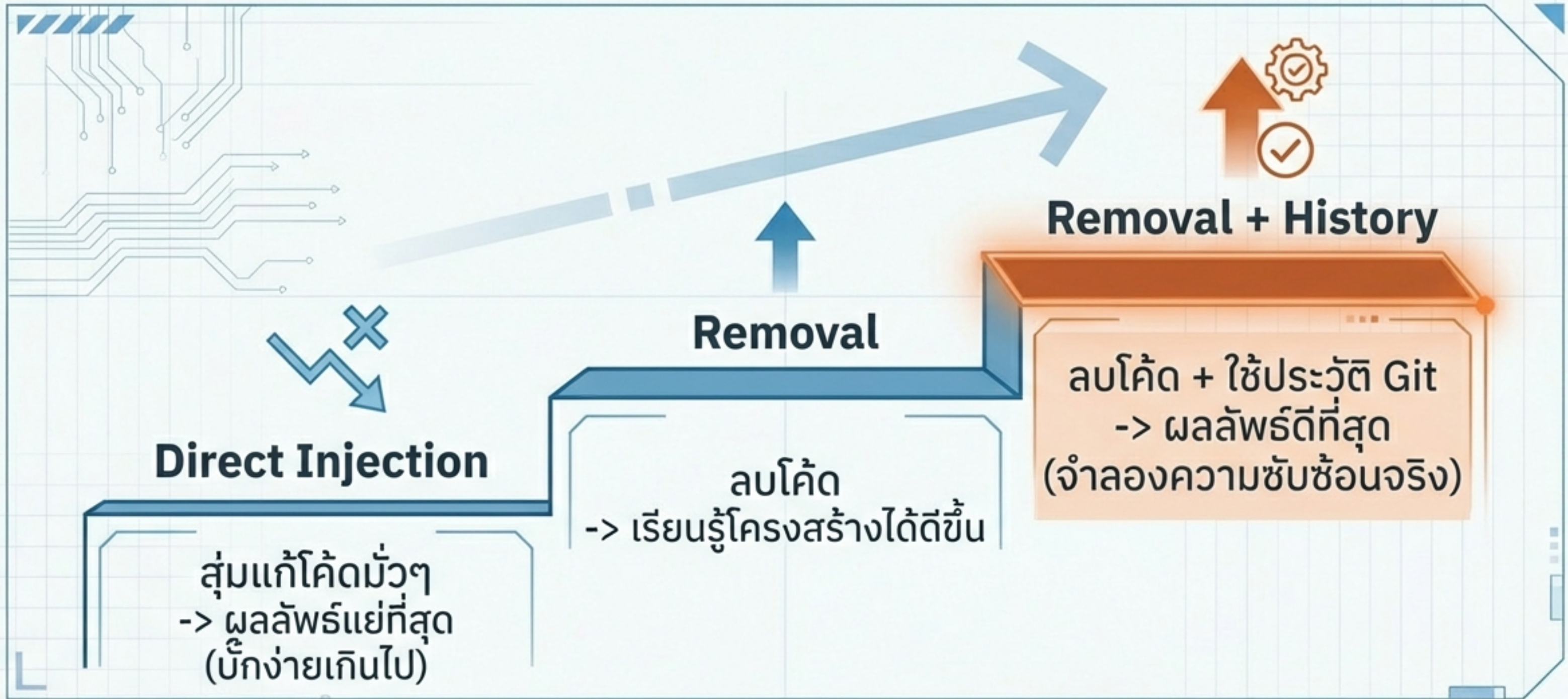
SSR (กราฟสีส้ม) มีพัฒนาการต่อเนื่องและอยู่เหนือ Baseline ที่ใช้ข้อมูลมนุษย์ตลอดการเทรน

Why Self-Play Matters?



Conclusion: กระบวนการ Active Learning ที่ต้องทำทั้ง ‘สร้าง’ และ ‘แก้’ คือหัวใจสำคัญของการเรียนรู้

Impact of Injection Strategy

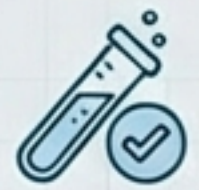


Limitations & The Future

Limitations (ข้อจำกัด)



ขาดโจทย์ภาษาธรรมชาติ
(Natural Language)
ในขั้นตอน Self-play

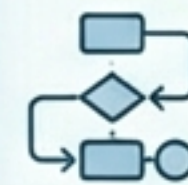


การตรวจสอบผลยังพึ่งพา
Unit Tests เป็นหลัก

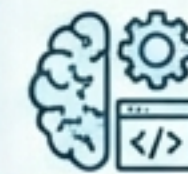
Future Work (ก้าวต่อไป)



Distribution Control: ควบคุม
การกระจายตัวของบีกให้ทั่วถึง
(Seeding)



Multi-step Tasks:
สร้างโจทย์ที่ซับซ้อนขึ้น
(ย้าย Version / สร้างฟีเจอร์ใหม่)



Specialized Agents: สร้าง AI
ที่เชี่ยวชาญเฉพาะ Codebase

Conclusion: ฤๅญแจสู่ Superintelligence

» SSR พิสูจน์แล้วว่า AI เรียนรู้ Software Engineering ได้จากการมีปฏิสัมพันธ์กับ Codebase โดยตรง



» ไม่จำเป็นต้องพึ่งพา ข้อมูลที่มนุษย์คัดกรอง

“From mimicking humans to exceeding them.”
(จากการเลียนแบบมนุษย์ สู่การก้าวข้ามขีดความสามารถของมนุษย์)